

3. Midiendo un qubit

Vamos a recrear en Qiskit el ejemplo 2.1 resuelto de forma algebraica. Nuestro estado inicial es $|\psi\rangle = \frac{1+i\sqrt{3}}{3}|0\rangle + \frac{2-i}{3}|1\rangle$. Si usamos la función `initialize`, podemos asignarle un estado arbitrario al qubit y luego calcular las probabilidades de cada uno de sus estados base

```
# -----  
# Recreando Ejercicio 2.6 Introduction to Classical and Quantum Computing. Thomas G. Wong  
# -----  
  
# Importamos las librerías necesarias  
from qiskit import QuantumCircuit  
from qiskit.quantum_info import Statevector  
from qiskit.visualization import plot_bloch_multivector  
import numpy as np  
  
# Creamos un circuito con 1 qubit  
qc = QuantumCircuit(1)  
  
# Definimos las amplitudes  
alpha = ( 1 + np.sqrt(3)*1j) / 3  
beta = ( 2 - 1j) / 3  
  
# Inicializamos el qubit con las amplitudes definidas  
qc.initialize([alpha, beta], 0)  
  
# Obtenemos el vector de estado del circuito  
state = Statevector.from_instruction(qc)  
  
# Probabilidades  
  
P0 = np.abs(state.data[0])**2  
P1 = np.abs(state.data[1])**2  
  
# Mostramos el vector numérico y sus probabilidades  
print("Vector de estado inicial: ", state.data)  
print(f"Prob(|0>): {P0:.4f}")  
print(f"Prob(|1>): {P1:.4f}")  
print(f"Suma de probabilidades: {P0 + P1:.4f}\n")
```

Salida:

```
Vector de estado inicial: [0.33333333+0.57735027j 0.66666667-0.33333333j]  
Prob(|0>): 0.4444}  
Prob(|1>): 0.5556}  
Suma de probabilidades: 1.0000
```

Como observamos anteriormente, la probabilidad del estado $|1\rangle$ es mayor que la probabilidad del estado $|0\rangle$, lo que sugiere que al medir el qubit este colapsará con más frecuencia a ese estado. En la práctica, la medición cuántica es un proceso probabilístico, por eso se realizan varias mediciones del mismo circuito y luego se cuentan los resultados para determinar a qué estado colapsa con mayor frecuencia.

En Qiskit, para poder medir un qubit es necesario agregar un bit clásico donde se almacene el resultado. Por esta razón, haremos un pequeño cambio cuando creamos el circuito y usaremos `QuantumCircuit(1, 1)` el primer argumento indica el número de qubits y el segundo el número de bits clásicos que registran las mediciones

```
# -----  
# Midiendo un qubit Ejercicio 2.6 Introduction to Classical and Quantum Computing. Thomas G. Wong  
# -----  
  
from qiskit import QuantumCircuit, transpile  
from qiskit.quantum_info import Statevector  
from qiskit_aer import AerSimulator  
from qiskit.visualization import plot_histogram  
import numpy as np  
import matplotlib.pyplot as plt  
  
# Creamos un circuito con 1 qubit y 1 bit clásico
```

```

qc = QuantumCircuit(1, 1)

# Definimos las amplitudes
alpha = ( 1 + np.sqrt(3)*1j) / 3
beta = ( 2 - 1j) / 3

# Inicializamos el qubit con las amplitudes definidas
qc.initialize([alpha, beta], 0)

# Obtenemos el vector de estado del circuito
state = Statevector.from_instruction(qc)

#Añadimos la medición del qubit al bit clásico
qc.measure(0,0)

# Simulador
sim = AerSimulator()

# Transpilamos y ejecutamos el circuito varias veces
tqc = transpile(qc, sim)
result = sim.run(tqc, shots=2000).result()

# Mostramos el vector numérico
print("\nVector de estado inicial: ", state.data)

# Conteos de resultados
counts = result.get_counts()
print("\nResultados de la medición:", counts)

# Histograma de resultados
plot_histogram(counts, figsize=(4, 3.5))

```

Salida:

```

Vector de estado inicial: [0.33333333+0.57735027j 0.66666667-0.33333333j]
Resultados de la medición: {'1': 1135, '0': 865}

```

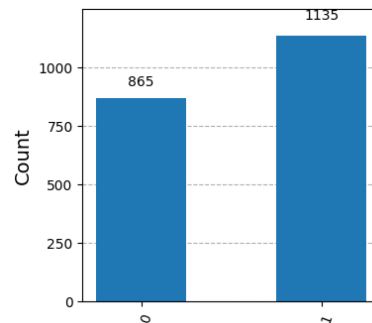


Figura 1: Frecuencia de resultados para $|0\rangle$ y $|1\rangle$

La figura 1 muestra que el resultado con más frecuencia es $|1\rangle$ con un conteo de 1135, por tanto $|\psi\rangle = |1\rangle$ después de la medición.

Referencias

- [1] D. McMahon. *Quantum Computing Explained*. IEEE Press. Wiley, 2008.
- [2] Thomas G. Wong. *Introduction to Classical and Quantum Computing*. University of Texas Rio Grande Valley, 2021.